Dayflow — Team Roles & Work Split (Frontend / Backend Divide)

Companion to: PRD.md, TRD.md, Navigation-Plan.md, Integration-Guide.md
Version: 1.0
Alternative to: Team-Roles.md (vertical-slice split)

This is a second way to divide the same 3-person team — by layer (frontend vs backend) instead of by module. Use this version if the team has clearer frontend/backend skill specialization than cross-module ownership; use Team-Roles.md instead if everyone is comfortable full-stack. Don't run both splits at once — pick one model for the project.

Split Overview

Member	Layer	Focus
Member A	Backend (Lead)	Auth, DB schema, all API modules — architecture owner
Member B	Frontend	Employee-facing screens + shared UI system
Member C	Frontend	Admin-facing screens + integration with backend APIs

Two frontend members instead of one is intentional: the UI surface (10+ routes across Employee and Admin per Navigation-Plan.md) is larger than one backend engineer can review meaningfully, while the API surface is compact enough for one person to own with fewer integration seams.

Member A — Backend (Lead / API Owner)

Owns everything in `apps/api` and the DB schema.

- **Auth: signup, email verification, login, JWT issuance/refresh, password hashing, rate limiting**
- **Role-based middleware (`requireAuth`, `requireRole`, `requireSelfOrAdmin`) — the contract both frontend members build against**
- **DB schema + Prisma migrations for all entities (User, Profile, Attendance, LeaveRequest, SalaryStructure, Notification)**
- **All REST endpoints (TRD §4): profile, employees, attendance, leave, payroll, notifications, reports**
- **Leave-approval → attendance-update transaction logic (the highest-risk business rule in the app)**
- **Email integration (verification + leave-decision emails)**
- **`packages/shared-types` — Zod schemas + inferred TS types, kept in sync with actual DB/API shape**
- **API error envelope convention**
- **Owns CI pipeline (lint/typecheck/test) and staging/prod deploy for the API**

Deliverable each phase: a working, documented endpoint set (with example requests/responses) that both frontend members can build against — ideally with mock data seeded so frontend isn't blocked waiting on real records.

Member B — Frontend: Employee Experience

Owns every Employee-role route in Navigation-Plan.md §1.

- `/dashboard` **— employee dashboard cards + activity feed**
- `/profile`, `/profile/edit` **— view/edit own profile**
- `/attendance`**, check-in/check-out UI**
- `/leave`, `/leave/new` **— apply for leave, view own leave status**
- `/payroll` **— read-only payroll view**
- `/notifications` **— notification list, read/unread state**
- **Signup/login/email-verification flow (shared entry point, but Member B builds it since it's the employee's first touchpoint)**
- **Consumes Member A's typed API client via TanStack Query hooks**
- **Co-owns the shared UI component library (** `apps/web/src/components/ui/` **) with Member C — Button, Card, Table, FormField built once, used by both**

Member C — Frontend: Admin Experience

Owns every Admin-role route in Navigation-Plan.md §1.

- `/admin/dashboard` **— employee list, attendance overview, pending approvals widget**
- `/admin/employees`, `/admin/employees/:id`, `/admin/employees/:id/edit`
- `/admin/attendance` **— all-employee attendance view**
- `/admin/leave`, `/admin/leave/:id` **— approve/reject with comments**
- `/admin/payroll`, `/admin/payroll/:id/edit`
- `/admin/reports` **— analytics/reports dashboard (Phase 1.5)**
- **"View as employee" read-only switch (reuses Member B's employee-dashboard components where possible — coordinate directly rather than duplicating screens)**
- **Co-owns the shared UI component library with Member B**

Coordination Points (this split's specific risk areas)

Compared to the vertical-slice split, this model concentrates all backend logic in one person and splits frontend by audience — so the friction points are different:

Risk	Mitigation
Member A becomes a bottleneck (both frontend members waiting on the same person)	**Member A ships API contracts (** `shared-types` **schemas + route signatures)** ***before*** **full implementation, so B and C can build against typed mocks while A finishes the real logic**

Risk	Mitigation
Member B and C build inconsistent UI (different button styles, table behavior)	Shared `components/ui/` library is designed together in Week 1, before either builds screens; no screen-specific one-off components for things the library should cover
Duplicate work on "View as employee" (Admin viewing an Employee's dashboard)	Member C reuses Member B's dashboard components in read-only mode rather than rebuilding — agree on this interface in Week 1
No one owns the leave→attendance business rule from the frontend side	It's entirely backend (Member A) — B and C just call `PATCH /leave/:id/decision` and re-fetch; neither frontend member needs to know the transaction internals

Suggested Sequencing

1. Week 1: Member A defines and shares API contracts (`shared-types` + route list with example payloads) for auth, profile, attendance. Members B & C set up the shared UI library together and build routing shell + role-aware layout.
2. Weeks 2–3: Member A implements auth + profile + attendance endpoints for real. Member B builds employee dashboard/profile/attendance screens against mocks, then swaps to real API as it lands. Member C starts admin employee-list/detail screens the same way.
3. Week 3–4: Member A builds leave module + approval transaction. Member B builds `/leave` + `/leave/new`. Member C builds `/admin/leave` + approval UI — this is the one pairing point worth a joint session since both are testing the same status-change flow.
4. Week 5: Member A builds payroll endpoints. Member B builds `/payroll` (read-only). Member C builds `/admin/payroll` (edit).
5. Week 6: Notifications, reports, responsive QA across both frontend surfaces, staging deploy.

Same phase order as Team-Roles.md — only the *ownership axis* changes, not the build order.